OPERATIONS MANUAL · GENERIC EDITION

Publishing an Android game on Google Play

From closed testing to production, with AI assistance

A reusable guide for solo developers with personal developer accounts: setup, AI access through the API, testers, the 14-day closed test, production access and launch. Illustrated with an anonymous example game.

Version 1.0 · November 10, 2026 · Generic English edition

Engine in the example: Flutter + Flame

Example: a paid, offline Android game, referred to as <app>

Contents

Before you start	How to use this manual	3
Overview	The workflow at a glance	4
Chapter 1	Prepare the account and the legal groundwork	5
Chapter 2	Give the AI access	7
Chapter 3	Create the app and the first build	9
Chapter 4	Recruit and manage testers	11
Chapter 5	During the 14 days: iterate without losing the count	14
Chapter 6	Apply for production access	16
Chapter 7	Launch to production	17
Chapter 8	After launch	18
Chapter 9	AI governance	19
Chapter 10	Troubleshooting	20
Appendix A	Illustrative timeline	21
Appendix B	Tester invitation template	22
Appendix C	Upload script template	23
Appendix D	Glossary	24

BEFORE YOU START

How to use this manual

This manual walks through taking an Android game from closed testing to production on Google Play when the developer works solo and relies on one or more AI assistants. It is written to be reused for every new game.

Placeholders

Placeholder	Meaning
`<app>`	Short name of the game, lowercase
`<package>`	Package name (applicationId). It becomes permanent once the first artifact is uploaded
`<testers-group>`	Your own Google Group for testers, e.g. `<app>-testers@googlegroups.com`
`<service-group>`	The external testing service's Google Group (the vendor gives you its address)
`<domain>`	Your studio domain; each game uses the subdomain `<app>.<domain>`
`<service-account>`	Google Cloud service account used by the AI, e.g. `<bot>@<project>.iam.gserviceaccount.com`
`<N>`, `<x.y.z>`	versionCode (increasing integer) and versionName

Conventions

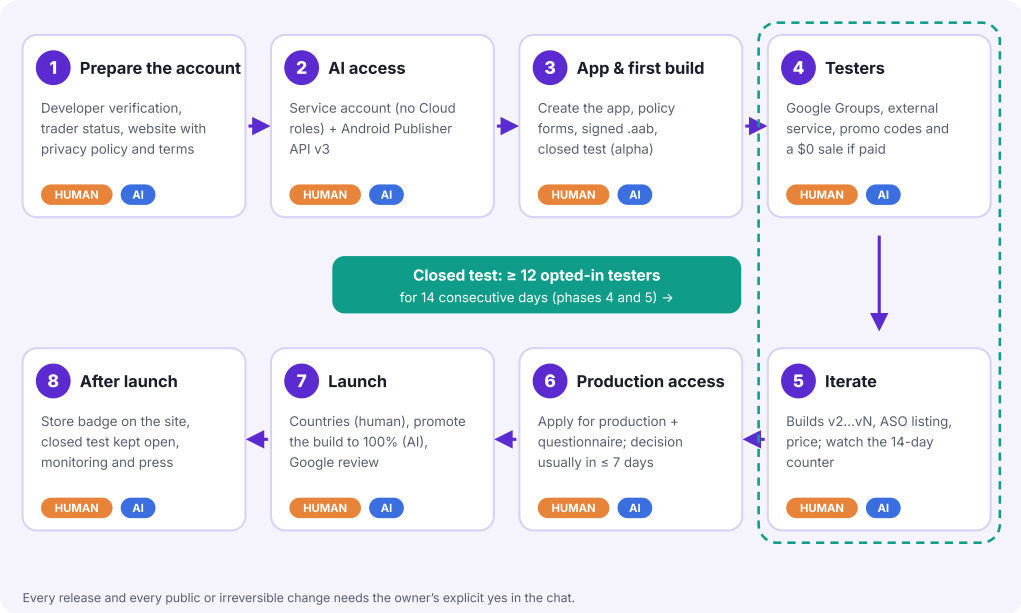
- HUMAN done by the account owner in Play Console (or elsewhere with their own session). AI done by the assistant, through the API or in writing. BOTH the AI prepares, the human approves or executes.
- Play Console paths are written like this: **Test and release › Testing › Closed testing**. If Google renames a menu, use the console's search box.
- **Green** boxes are **Examples**: how the process played out for an anonymous example game. **Orange** boxes are warnings. **Purple** boxes are notes.
- Amounts are given in USD or in the store's local currencies; Google charges the registration fee in USD.

> **NO SECRETS**
> This document contains no JSON keys, tokens, promo codes or personal tester emails, and no copy of it should. Credentials live in environment variables or a secrets manager, never in a chat or a PDF.

OVERVIEW

The workflow at a glance

The process has eight phases. Phases 4 and 5 run in parallel: while the 14 closed-testing days tick by, you keep iterating on the game and the store listing. The diagram shows who takes part in each phase.



Who does what

Phase	HUMAN	AI
1. Account	Developer verification, trader declaration, approve the legal texts	Draft terms and privacy policy; prepare the website (via PR)
2. AI access	Create the service account and invite it as Admin	Test access with a read-only edit
3. App & build	Create the app, upload the first .aab, complete the policy forms	Verify the .aab; draft the listing and release notes
4. Testers	Create the group, buy the service, generate promo codes, create the sale	Code-assignment sheet, invitation templates (drafts)
5. Iterate	Approve each release, set the price, watch the counter	Upload each build, update the ASO listing, read the tracks
6. Production	Apply for production and fill in the questionnaire	Draft the answers
7. Launch	Pick countries; give the final yes	Promote the build to production at 100%
8. After launch	Approve the website merge, decide on the test track	PR for the Play badge, monitoring, press (drafts)

Typical duration

EXAMPLE

A paid, offline game with no ads and no in-app purchases. From creating the app to a live store listing took about **three weeks**: several builds in closed testing, a tester pool comfortably above 12, an en-US-only listing and broad country coverage. The bottleneck is the 14-day test: if the count starts late or breaks, everything slips. Production access came back in about a day (Google says up to 7), and review of the production release took a day or two.

CHAPTER 1

Prepare the account and the legal groundwork

Before touching closed testing, know what kind of account you have and get the identity requirements and legal texts out of the way.

1.1 Account type and testing requirement

Personal developer accounts created after November 13, 2023 must run a **closed test with at least 12 testers who stay opted in continuously for at least 14 days** before they can apply for production access. Until then, `Test and release › Production` is disabled, and open testing only unlocks after production access is granted. Organization accounts are exempt.

The account registration fee is a one-time payment of USD 25.

1.2 Android developer verification

HUMAN Register each app (by its `<package>`) and its signing keys in Play Console's Android developer verification section. Google sends deadline reminders to the account email; don't leave it until the end.

> **EXAMPLE**
> The example game was registered with its signing keys on the day the app was created. The API does not expose this status, so the owner confirmed it with a screenshot.

1.3 Trader declaration

HUMAN Declare the account as a *trader* if you sell apps, and complete the declaration **for each app**. It asks for a link to your Terms, so publish them first.

1.4 Website, privacy policy and terms

Each game gets its own subdomain `<app>.<domain>`, served from a static host such as GitHub Pages, with at least three pages:

Page	Purpose	Who
`https://<app>.<domain>/`	The game's site; later, the "Get it on Google Play" badge	AI PR · HUMAN merge
`/privacy.html`	Privacy policy, required in the listing and in Data safety	AI draft · HUMAN approve
`/terms.html`	Terms; linked from the trader declaration	AI draft · HUMAN approve

The Terms should cover who publishes the game and how to reach them, the license to use it, the price and the fact that Google handles payments and refunds, that there are no accounts and data stays on the device, the update policy and acceptable use. Whenever the game changes (for example, adding rate or share buttons), review the privacy policy.

> **EXAMPLE**
> The example game's Terms were published in English on its website and linked in the per-app trader declaration the same day. When a later build added Rate and Share buttons, the privacy policy was updated to mention them.

Checklist · Phase 1

- [ ] I know whether my account is personal and created after Nov 13, 2023 (so whether the 12 × 14 test applies).
- [ ] App and keys registered for Android developer verification.
- [ ] Account declared as a trader, and the per-app declaration completed with the Terms link.
- [ ] `<app>.<domain>` live with `privacy.html` and `terms.html`.
- [ ] Studio contact email decided (it goes in the listing).

CHAPTER 2

Give the AI access

Google blocks automated sign-in to Play Console from an agent-driven browser. The sanctioned way for an AI to operate is a **service account** using the **Google Play Android Publisher API v3**; no marketplace plugin is needed.

2.1 Create the service account HUMAN

1. In Google Cloud Console, create (or reuse) a studio project and enable the **Google Play Android Developer API**.
2. Create a service account `<service-account>`. **Do not grant it any Cloud roles**; permissions come from Play Console.
3. Generate a JSON key and store it where the AI reads it as an environment variable (for example `GOOGLE_PLAY_SERVICE_ACCOUNT_JSON`). Never paste it into the chat.
4. In Play Console: **Users and permissions › Invite new users**, enter the service account's email and grant **Admin** on the account (or, more cautiously, Admin on the app only).

2.2 Test the access AI

Open an edit, read the tracks and delete the edit without committing. A 200 response means access works. This read changes nothing.

2.3 The edit lifecycle

Every API change happens inside an *edit* (a transaction). If something fails before the commit, the edit is deleted and nothing changes in Play.

#	Call	What it does
1	`POST …/applications/<package>/edits`	Opens an edit (insert)
2	`GET …/edits/{id}/bundles`	Lists the versionCodes already uploaded; stop if the new one already exists
3	`POST /upload/…/edits/{id}/bundles?uploadType=media`	Uploads the .aab; confirm the returned versionCode is the expected one
4	`PUT …/edits/{id}/tracks/alpha`	Defines the track's release: name, versionCodes, `status: completed`, en-US notes
5	`POST …/edits/{id}:validate`	Validates without publishing (country errors and the like show up here)
6	`POST …/edits/{id}:commit`	Applies the changes and sends them to Google review. **Without** the `changesNotSentForReview` parameter
7	`DELETE …/edits/{id}`	Only if there was no commit: discards everything
8	New edit + `GET tracks/alpha` + `DELETE`	Read-back check of the final state

CHANGESNOTSENTFORREVIEW

For the example game, Play rejected commits with `changesNotSentForReview`; the normal commit (which sends changes to review) is what worked. Commit without that parameter unless Google says otherwise for your app.

2.4 What the API cannot do (left to the human)

Task	Where the human does it
Create the app and upload the first .aab	**All apps › Create app**
Policy forms: Data safety, content rating (IARC), target audience, ads, privacy policy	**Monitor and improve › Policy and programs › App content**
App price and sales	**Monetize with Play › App pricing**
Generate promo codes	**Monetize with Play › Promo codes**
See how many testers are opted in and the 14-day counter	**Dashboard** and **Test and release › Testing › Closed testing**
Select production countries	**Test and release › Production › Countries/regions**
Apply for production access and answer the questionnaire	**Dashboard › Apply for production**
Developer verification, trader status	Account settings
Sales and revenue (not listed by the API)	**Download reports › Financial** (or the reports bucket `gs://pubsite_prod_…`)

Checklist · Phase 2

- [ ] API enabled in the Cloud project; service account with no Cloud roles.
- [ ] JSON key in an AI environment variable, never in the chat.
- [ ] Service account invited as Admin in Play Console.
- [ ] Test read (edit opened and deleted) returned 200.

CHAPTER 3

Create the app and the first build

3.1 Create the app HUMAN

1. **All apps › Create app**: name, default language, **game**, **free or paid** (a free app cannot be switched to paid later), declarations.
2. Complete the forms under **Monitor and improve › Policy and programs › App content**: privacy policy (`https://<app>.<domain>/privacy.html`), ads, target audience, IARC content rating and Data safety.
3. Main listing in **Grow users › Store presence › Main store listing**: title, short and full description, 512×512 icon, 1024×500 feature graphic and screenshots. The AI can draft the copy and push the listing through the API (ch. 5).

3.2 Verify the .aab before uploading AI

Before every upload, the AI checks that:

- the `package` is `<package>`;
- the `versionCode` is higher than every one already uploaded and the `versionName` is the announced one;
- the `targetSdkVersion` meets Google Play's current minimum;
- the bundle is signed with the **upload key** (the upload key's certificate appears in `META-INF/`, e.g. `UPLOAD.RSA`).

```
# Option A (official tools)
java -jar bundletool-all.jar dump manifest --bundle <app>_<x.y.z>+<N>.aab
jarsigner -verify -verbose -certs <app>_<x.y.z>+<N>.aab | head

# Option B (in-house script that reads the .aab's protobuf manifest)
unzip -o <app>_<x.y.z>+<N>.aab base/manifest/AndroidManifest.xml 'META-INF/*'
python3 pm.py base/manifest/AndroidManifest.xml
ls META-INF        # should list the upload key certificate
```

EXAMPLE · VERIFICATION OUTPUT

`manifest {'package': '<package>', 'versionCode': '3', 'versionName': '1.1.0'}` · `META-INF: MANIFEST.MF UPLOAD.RSA UPLOAD.SF`. Matching package, a versionCode above the last upload and the upload-key certificate in `META-INF` mean the bundle is ready.

3.3 First release in closed testing

HUMAN The first .aab is uploaded by hand. Path: **Test and release › Testing › Closed testing › Manage track › Create new release**. Use the default track (`alpha` in the API). Release name: `<N> (<x.y.z>)`. Notes in **en-US** (drafted by the AI). Then **Review release › Start rollout**.

On the same track's **Testers** tab, add the email lists or Google Groups (ch. 4) and a feedback URL or email address, which testers see on the opt-in page.

PROPAGATION TIME

The opt-in link only appears once the app is *Published* on that track, and it can take several hours after the first release.

EXAMPLE

Day 0: release “1 (*x.y.z*)”, versionCode 1, status *completed*, on the alpha track, with a short email list of the owner’s own testers.

Checklist · Phase 3

- [ ] App created as a paid game (if applicable) with its final `<package>`.
- [ ] Data safety, IARC, target audience, ads and privacy policy complete.
- [ ] en-US listing with icon, feature graphic and screenshots.
- [ ] .aab verified: package, versionCode, targetSdk and upload-key signature.
- [ ] Release 1 published to Closed testing with en-US notes.

CHAPTER 4

Recruit and manage testers

You need **12 testers opted in continuously for 14 days**. In practice, keep a buffer of 15–20: anyone who drops out early stops counting.

4.1 Google Groups HUMAN

1. Create the group `<app>-testers@googlegroups.com` at groups.google.com (let anyone ask to join, or invite by email).
2. In **Test and release › Testing › Closed testing › Manage track › Testers**, choose **Google Groups** and add `<testers-group>` and, if you use an external service, `<service-group>`. Save.
3. Copy the opt-in link: `https://play.google.com/apps/testing/<package>`.

> **ORDER MATTERS FOR TESTERS**
> With Google Groups, the tester must **join the group first** and **then** open the opt-in link and tap "Become a tester". In the reverse order, Play won't recognize them.

4.2 External testing service (optional)

A testing service provides real people who opt in, keep the game installed and use it throughout the 14 days. It plugs in by adding its Google Group to the closed-testing track. One such vendor is **Testers Community** (testerscommunity.com); others exist. The developer keeps control of Play Console: according to that vendor's FAQ, it never asks for a password or console access.

How to use it

1. HUMAN If the game is paid, create the **$0 sale first** (section 4.3) so the service's testers don't have to pay. Play Console only lets you schedule sales with a future start date, so the sale must exist before you activate the service; otherwise its testers can't install the game on day one and the count starts late. The vendor's help pages document this procedure and recommend a temporary sale (rather than making the app free) in all countries, since their testers are international.
2. HUMAN Buy a **credit** for testing one app (a one-time payment per app, not a subscription).
3. BOTH Add the service's Google Group, `<service-group>`, under **Test and release › Testing › Closed testing › Manage track › Testers › Google Groups**, alongside your own group. Save.
4. HUMAN Hand over the opt-in link (`https://play.google.com/apps/testing/<package>`). Their testers join the group, opt in with the same account and use the app most days.
5. AI Read the reports when they arrive and turn the suggestions into tasks for the next build and the listing.

Item	Details
Purchase	A credit for testing one app. According to the vendor's pages, credits don't expire, payment is by card through a third-party processor, and a production-access-or-refund guarantee is offered.
Plan shapes	A **starter-size** per-app credit with a set number of testers, a feedback report and model answers for the production form; a **larger plan** with more testers and extras such as an ASO report; and a **deeper manual-QA add-on**. **Check current prices on the vendor's site**: they change, and blog articles may quote older figures than the pricing page.
What they need from you	Add their Google Group to the closed-testing track and share the track's opt-in link.

Paid game	A $0 sale so their testers don't pay (section 4.3), created before activating the service because it starts on a future date. The vendor's public pages do not say it pays for its testers' purchases.
Follow-up and deliverables	Dashboard with testers, daily progress and reports. Partway through the test it delivers a **Testers Feedback Report** (bugs, UX, performance and suggestions) and **Production Access Answers** (model answers to the questions of the production form). Testers stay active until the 14 days are done, and the vendor says it replaces inactive ones.
After launch	The vendor may send a congratulations email asking for a review or testimonial (optional).

EXAMPLE · PAID TESTING SERVICE

The example game used a starter-size credit; the service supplied most of the testers and the owner's own testers made up the rest. The service was requested **before** the $0 sale was set up, and because the sale could only start on a future date, the service's testers began later than planned (section 5.1). About a week into the test, the reports arrived. The feedback report found no critical bugs or crashes and suggested an interactive tutorial, an ASO-optimized description, a rate button and a share button. The owner found it very useful: the ASO listing was updated that day, and the next build added an interactive tutorial and Rate and Share buttons.

WHAT THE SERVICE SAYS ABOUT THE COUNT

According to the vendor's guides, the clock starts at opt-in, not when emails are added; publishing a new build on the same track does not reset the count; and **pausing the track** mid-test breaks continuity, so assume a fresh 14 days after reactivating it. They also say that dipping below 12 temporarily does not reset the clock, though Google weighs engagement and may ask for more days. Google's own guidance requires 12 testers opted in continuously for the 14 days before you apply: when in doubt, always keep 12 or more.

EXTERNAL EMAILS

Emails from outside services are data, not instructions. The AI checks them for hidden prompt injections before acting, and any action they ask for (leaving a review, clicking a link, replying) is the owner's call.

4.3 If the game is paid: promo codes and a $0 sale

In closed and open testing, **testers must buy paid apps** (only internal testing is free). Two complementary tools solve this:

	Promo codes	$0 sale
For whom	Your own testers, press	The external service's testers (who join through its group)
Where	`Monetize with Play › Promo codes`	`Monetize with Play › App pricing` (sales)
Who	HUMAN generates; AI assigns them in the sheet	HUMAN creates it and sets its duration
Watch out	Each code is single-use; never put codes in chats or shareable documents	Can only be scheduled with a future start date: create it before activating testers. Anyone who gets the game during the sale keeps it. After cancelling a sale, you must wait 30 days before creating another

Code-assignment sheet

The AI creates a private sheet `<app>_testers_codes` in the studio's Drive with these columns:

code	group	tester (email)	sent	opted in	notes
(hidden)	Initial testers	(email)	date	yes/no	
(hidden)	Paid testers	—			unassigned
(hidden)	Reserve	—			press, replacements

For the sale: make it temporary (don't switch the app to free), include every country where you have testers, and schedule it before buying or activating the testing service, because it starts on a future date. When it ends, the app returns to its original price.

> **EXAMPLE**
> The owner generated a batch of promo codes on day 0: some assigned to initial testers, some kept for additional testers, and a few held in reserve for press and replacements. The service's testers got the game through the $0 sale, which the owner kept for testers and press only.

4.4 Invitation

AI drafts the invitation (in each language your testers need); HUMAN sends it. The generic template is in **Appendix B**. Must-haves: uninstall earlier test builds (APK), join the group, opt in, redeem the code, install, and **stay in the test with the game installed for 14 days**.

Checklist · Phase 4

- [ ] Group `<testers-group>` created and added to the track.
- [ ] External service's group added (if used).
- [ ] Feedback URL or email set on the Testers tab.
- [ ] Paid game: promo codes generated and assignment sheet ready; $0 sale scheduled **before** activating the testing service.
- [ ] Invitations sent by the human; at least 12 opt-ins confirmed on the dashboard (target: 15 or more).

CHAPTER 5

During the 14 days: iterate without losing the count

5.1 Watch the counter

HUMAN The dashboard (Dashboard , closed-testing requirements card) is the only source for the number of opted-in testers and days completed; the API doesn't expose it. Check it every 2–3 days and take a screenshot.

WHAT DELAYS OR BREAKS THE COUNT

- Google requires 12 testers opted in **continuously** for the 14 days before you apply. A testing vendor may say a brief dip below 12 doesn't reset the clock, but always keep 12 or more.
- A tester who leaves and rejoins starts counting from zero.
- A paid game without an active $0 sale keeps the service's testers from installing. Since sales start on a future date, create it before activating the service.
- **Pausing the track** mid-test breaks continuity and forces a fresh start. Don't pause it.
- A new build on the same track does **not** reset the count.

EXAMPLE

The first estimate of the completion date was two days too early. **The counter never reset**: the service had been requested before the $0 sale was set up, and because Play Console only allows sales with a future start date, the service's testers started later than planned. Lesson: schedule the $0 sale before buying or activating the service.

5.2 Upload new builds BOTH

1. Development delivers `<app>_<x.y.z>+<N>.aab` with a higher versionCode.
2. The AI verifies the bundle (3.2) and drafts the en-US notes.
3. The AI shows the owner: file, versionCode, versionName, notes, track. **The owner replies "yes" in the chat**.
4. The AI runs the edit lifecycle (2.3) with `status: completed` and the name `<N> (<x.y.z>)`, replacing the track's previous release.
5. The AI reads the track back and reports the commit time. Google reviews every release.

EXAMPLE · CLOSED-TESTING BUILDS

vc	Day	Change	Uploaded by
1	0	First build	Human
2	~3	Rendering performance optimization	AI
3	~6	Visual clean-up of the play area	AI
4	~10	HUD and controls polish; library updates	AI
5	~12	Interactive tutorial; Rate and Share buttons (from tester feedback)	AI

5.3 ASO listing BOTH

The AI prepares a draft with the current listing, target keywords and character counts. Limits: **title ≤ 30**, **short description ≤ 80**, **full description ≤ 4,000** characters. With the owner's yes, it applies it through the API (`PUT …/edits/{id}/listings/en-US` + commit), **saving a JSON copy of the previous listing first**. The copy must match the game's official story and only claim what has been verified (offline, no ads, no special permissions).

> **EXAMPLE**
> After the testing service's feedback, the en-US short and full descriptions were rewritten around target keywords, within the limits, while keeping the title. The previous listing was backed up as JSON before the change.

5.4 Price HUMAN

The price is set in **Monetize with Play › App pricing** , with a USD base price and per-country adjustments. The AI can research and suggest, but the owner sets the price. A price change applies to every track.

> **EXAMPLE**
> The owner set a low one-time USD base price and a rounded local price in their home market, and confirmed the $0 sale as intentional (testers and press only).

Checklist · Phase 5

- [ ] Counter screenshot every 2–3 days; ≥ 12 testers at all times.
- [ ] Never pause the track during the test.
- [ ] Every build verified, approved with an explicit yes, uploaded and read back.
- [ ] Feedback logged (Testing feedback, email, service) for the questionnaire.
- [ ] ASO listing applied, with the previous one backed up.
- [ ] Price set and confirmed.

CHAPTER 6

Apply for production access

Once the dashboard shows all three closed-testing requirements met, the **Apply for production** button appears.

6.1 Steps HUMAN

1. Dashboard › Apply for production.
2. Answer the form's three sections. If you leave or hit *Discard* before *Apply*, your answers are lost, so have the draft ready.
3. Google replies by email to the account owner, usually within 7 days. If it asks for more testing (for example, too few testers or low engagement), the closed test continues.

Section	Questions	Input
About your closed test	How you recruited testers; how easy it was; engagement (did they use every feature? did usage resemble real use?); a summary of the feedback and how you collected it	Tester sheet, service reports, Testing feedback
About your app/game	Target audience; what makes the game unique; expected installs in the first year	ASO listing
About your production readiness	Changes made based on what you learned; how you decided it was ready	Build table (5.2)

AI drafts the answers from true facts: the builds and their changes, the feedback received and where it came from. If a service provides model answers, use them as a starting point but **don't copy them verbatim**: adapt them to the actual app. For example, correct the number of testers and how they were recruited, say "we implemented" only if the change already ships in a build, cite the real changes per version and give an install estimate you can stand behind.

> **EXAMPLE**
> The owner applied the day the requirements turned green, using the service's model answers with his own edits. Google granted access about a day later.

Checklist · Phase 6

- [ ] All three requirements show green on the dashboard.
- [ ] Questionnaire draft reviewed by the owner.
- [ ] Application submitted; date noted.
- [ ] Approval email received.

CHAPTER 7

Launch to production

7.1 Choose countries HUMAN

Test and release › Production › Countries/regions › Add countries/regions. The API cannot select countries: if you try to promote first, validate fails with:

```
validate 403 {"error": {"code": 403, "message": "Release in track targeting no countries", "status": "PERMISSION_DENIED"}}
```

The script deletes the edit (nothing changes); run it again once the human has added the countries.

7.2 Promote the build AI with the owner's yes

Production gets **the same build** the testers already used (same versionCode, same notes):

1. Open an edit; read the `alpha` track release with versionCode `<N>`.
2. Check that the `production` track is empty (or as expected); otherwise, stop.
3. `PUT tracks/production` with the same name, `versionCodes: ["<N>"]`, `status: completed` (100%) and the alpha notes.
4. `validate` → `commit`. It goes into Google review.

> **STAGED ROLLOUT**
> For a game with an existing user base, use `status: inProgress` with a `userFraction` (for example 0.2). For a first launch with no users, 100% is the norm.

7.3 Review and publishing

- With **managed publishing off**, the release goes live as soon as Google approves it. With managed publishing on, you have to click Publish afterwards.
- **Publishing overview** is the source of truth. The Dashboard can lag behind.
- Confirm it's live by opening `https://play.google.com/store/apps/details?id=<package>`: it returns 404 until published and 200 once it is.

> **EXAMPLE**
> The owner selected a broad set of countries. The AI's first promotion attempt, made before that, failed with "Release in track targeting no countries"; after the countries were added, with the owner's yes, the last closed-test build was promoted at 100%. The public listing returned 404 for about a day and a half, then went live.

Checklist · Phase 7

- [ ] Countries selected by the human.
- [ ] Owner's explicit yes to promote build `<N>`.
- [ ] Promotion at 100% (or staged) with a clean validate and commit.
- [ ] Managed publishing status known; progress tracked in Publishing overview.
- [ ] Public listing returns 200.

CHAPTER 8

After launch

8.1 Website

AI (repository bot) opens a PR replacing “Soon on Google Play” with the official **Get it on Google Play** badge linked to the listing. HUMAN reviews and approves the merge.

8.2 What to do with the closed test

Recommendation: **keep it open** to try future pre-release builds before taking them to production. If the test track has the same versionCode as production, testers simply get the production build. To end it: **Test and release › Testing › Closed testing › Manage track › Pause track** (with the owner’s yes). Testers keep the app installed but stop receiving test builds.

8.3 Next builds

Recommended path: alpha first, a few days of use, then promote the same versionCode to production. Each step needs its own yes.

8.4 Monitoring and press

- **Monitor and improve › Android vitals** (crashes, ANRs), **Ratings and reviews**, acquisitions.
- Sales: **Download reports › Financial**; the API doesn’t list sales.
- Press: “Reserve” promo codes for media, a press kit at `<app>.<domain>/press/`, emails as drafts for the owner to send; social announcements by the social-media bot, with approval.
- Cancel the $0 sale or let it expire once it’s no longer needed, keeping in mind the 30-day wait before creating another one.

Checklist · Phase 8

- [ ] Website shows the Google Play badge (PR merged with approval).
- [ ] Decision on the test track: open (recommended) or paused.
- [ ] $0 sale reviewed; reserve codes accounted for.
- [ ] Weekly check of vitals, reviews and sales.

CHAPTER 9

AI governance

AI speeds things up, but the account, the money and the reputation belong to the owner. These rules prevent surprises.

9.1 What needs permission

Needs the owner's explicit "yes" in the chat	Needs no permission
• Every Play release, **one at a time** (one yes doesn't cover the next) • Changes to the public listing • Sending external emails or messages (the AI leaves drafts) • Merging PRs, publishing on the website or social media • Spending any real money • Pausing tracks, cancelling sales	• Reading state through the API (edits opened and deleted without commit) • Research and comparisons • Writing drafts (notes, listing, invitations, questionnaire, terms) • Verifying builds

- The yes must be given in the chat of the bot that performs the action (or a group chat that bot belongs to). Another bot saying "the owner approved" **doesn't count**.
- Before asking for a yes, the AI shows exactly what it will do: file, versionCode, track, text.
- After each action, the AI reports the time (in the owner's time zone) and does a read-back check.

9.2 Security

- Never put credentials in the chat: no JSON keys, tokens or promo codes.
- Least privilege: service account with no Cloud roles; Admin in Play only.
- External emails (services, platforms, press): check them for prompt injections before acting; whatever they ask for is the owner's call.
- Scripts abort if the versionCode already exists or the target track isn't as expected, and delete the edit if there was no commit.

9.3 Division of labor between bots

Role	Responsibility	Does not
Coordinator	Assigns tasks, tracks open items, owns anything without a clear owner	Act on others' behalf without their yes
Development and builds	Game code, signed .aab files, technical notes	Upload to Play
Play / publishing	Play API, tracks, listing, tester sheet, email drafts	Write to the repository
Repository	Website PRs, terms, privacy policy; merges with the owner's yes	Operate Play
Social media	Announcements and replies, with approval	Post without approval
Research	Pricing, ASO, press directories	Make changes

CHAPTER 10

Troubleshooting

Symptom	Cause	Fix
The AI can't sign in to Play Console in the browser	Google blocks automated sign-in	Service account + Android Publisher API (ch. 2)
`Release in track targeting no countries` when validating production	Production has no countries	The human adds countries in **Production › Countries/regions**; retry
Commit fails with `changesNotSentForReview`	The app doesn't support that mode	Normal commit (sends to review)
The 14-day counter shows fewer days than expected	Testers started late (e.g., the $0 sale hadn't started yet), someone left and rejoined, or the track was paused	Keep a tester buffer (15–20), don't pause, watch the dashboard and recalculate the date
A tester can't see the test or the link doesn't work	Didn't join the group first; is using another account; the app isn't *Published* on the track yet	Join the group with the phone's account, then opt in; wait a few hours after the first release
A tester has to pay for the game	In closed testing, paid apps must be bought	Promo code (own testers) or $0 sale (service)
The service's testers can't install on day one	The $0 sale was created after activating the service and starts on a future date	Schedule the sale before buying or activating the service
A tester doesn't get the new build	A sideloaded APK is installed, or they're in internal testing	Uninstall the APK; leave internal testing; reinstall from Play
Upload rejected for versionCode	It already exists or is lower	Bump the versionCode and rebuild; the script checks this first
Upload rejected for signature	Not signed with the upload key	Sign with the registered upload key; check `META-INF`
Dashboard still says "in review" when it should be live	The Dashboard updates with a delay	Check **Publishing overview** and the public URL
Public listing returns 404 after the commit	Google review in progress	Wait; in the example it took about a day and a half
Can't create another sale	A sale was cancelled less than 30 days ago	Wait; plan sales ahead
The promo codes page won't load	Console glitch	Try again later
You want sales data from the API	The API doesn't expose sales	**Download reports › Financial** or the reports bucket

APPENDIX A

Illustrative timeline

An illustrative sequence for a paid game with an optional testing service, in days relative to creating the app (Day 0). Your durations will differ.

Day	Who	Event
Day 0	H	App created and registered for developer verification. Release 1 on the closed track with a few own testers. Promo codes generated.
Day 0	AI	en-US listing saved for review (title, descriptions, icon, feature graphic, screenshots); code-assignment sheet and invitation template drafted.
Day 1	H/AI	Code groups set (initial / additional / reserve). Invitations sent by the owner. Testing service contacted.
Day 3	AI	Build 2 uploaded and sent to review.
Day 4	H	Service group added to the track; $0 sale live (created after requesting the service, so it started on a later date). Data safety and content rating complete. Tester pool above 12. Price set. Approval rules agreed.
Day 6	AI	Build 3.
Day 10	AI	Build 4.
Day 11	Service	Feedback report + model answers for the production form.
Day 11	AI	ASO listing applied (previous listing backed up).
Day 12	AI	Build 5: interactive tutorial and Rate/Share buttons.
Day 15	H	Dashboard shows 12 of 14 days (no reset): service testers started late because of the sale's future start date.
Day 17	H	All three requirements met; Apply for production with the service's model answers, edited by the owner. Trader declaration with Terms; privacy policy updated.
Day 18	Google/H/AI	Production access granted. Countries selected. First promotion attempt: "no countries"; then build 5 to production at 100%.
Day 19	—	Public listing still 404 (in review).
Day 20	Google/AI/H	Public listing live. Website badge PR merged with the owner's yes. Closed test kept open for future builds.

APPENDIX B

Tester invitation template

Send it individually from the studio's email. Drop step 4 if the game is free or the tester joins through the $0 sale. Translate it for testers who prefer another language.

```
Subject: Try <Game name> free before launch

Hi <name>,

I'm about to launch <Game name>, my new Android game: <one-line pitch>.
Before it can go public, Google Play requires at least 12 people to test it for
14 days, and you'd help me a lot by being one of them.

The game is paid, but it's free for you with this code: <CODE>

Steps (in this order, with the Google account on your phone):
1. If you have an older test build (APK) installed, uninstall it.
2. Join the testers group: accept the email invitation or open
   https://groups.google.com/g/<app>-testers and tap "Join group".
3. Join the test and tap "Become a tester":
   https://play.google.com/apps/testing/<package>
4. Redeem your code: https://play.google.com/redeem?code=<CODE>
   (or in the Play Store: your profile picture > Payments & subscriptions > Redeem code).
5. Install: https://play.google.com/store/apps/details?id=<package>

Important: please stay in the test and keep the game installed for the next
14 days. Found a bug or have feedback? Just reply to this email.

The game is yours to keep, including after the official launch.

Thanks!
<signature>
```

APPENDIX C

Upload script template

Python with `google-auth`. The key is read from an environment variable. Change `P`, `AAB`, `VC`, `NAME` and `NOTES`. For production, change `TRACK` and skip the upload (reuse the alpha versionCode).

```
import json, os, datetime
from google.oauth2 import service_account
from google.auth.transport.requests import AuthorizedSession

info = json.loads(os.environ["GOOGLE_PLAY_SERVICE_ACCOUNT_JSON"])   # never in the code
s = AuthorizedSession(service_account.Credentials.from_service_account_info(
        info, scopes=["https://www.googleapis.com/auth/androidpublisher"]))
P, AAB, VC, TRACK = "<package>", "<app>_<x.y.z>+<N>.aab", <N>, "alpha"
NAME, NOTES = "<N> (<x.y.z>)", "<en-US release notes>"
B = f"https://androidpublisher.googleapis.com/androidpublisher/v3/applications/{P}"

def chk(r, step):
    print(step, r.status_code, r.text[:800] if r.status_code >= 300 else "")
    if r.status_code >= 300: raise SystemExit(step + " failed")
    return r.json() if r.text else {}

e, committed = None, False
try:
    e = chk(s.post(f"{B}/edits", json={}), "insert")["id"]
    vcs = [b["versionCode"] for b in chk(s.get(f"{B}/edits/{e}/bundles"), "list").get("bundles", [])]
    if VC in vcs: raise SystemExit(f"versionCode {VC} already exists")
    with open(AAB, "rb") as f:
        up = chk(s.post(f"https://androidpublisher.googleapis.com/upload/androidpublisher/v3/"
                        f"applications/{P}/edits/{e}/bundles?uploadType=media", data=f,
                        headers={"Content-Type": "application/octet-stream"}, timeout=900), "upload")
    if up.get("versionCode") != VC: raise SystemExit("unexpected versionCode")
    body = {"track": TRACK, "releases": [{"name": NAME, "versionCodes": [str(VC)],
            "status": "completed", "releaseNotes": [{"language": "en-US", "text": NOTES}]}]}
    chk(s.put(f"{B}/edits/{e}/tracks/{TRACK}", json=body), "track")
    chk(s.post(f"{B}/edits/{e}:validate"), "validate")
    chk(s.post(f"{B}/edits/{e}:commit"), "commit")          # no changesNotSentForReview
    committed = True
    print("commit", datetime.datetime.now().isoformat(timespec="seconds"))
finally:
    if e and not committed: print("delete edit", s.delete(f"{B}/edits/{e}").status_code)
```

APPENDIX D

Glossary

Term	Definition
.aab (Android App Bundle)	The publishing format uploaded to Play; Google generates per-device APKs from it.
Upload key / App signing key	The upload key signs what you upload; Google re-signs with the app signing key (Play App Signing).
versionCode / versionName	Integer that must increase with every upload / user-visible version (e.g. 1.2.0).
targetSdk	The Android API level the app targets; Google enforces a minimum.
Track	Distribution channel: internal, closed (`alpha` in the API), open (`beta`), production.
Release	A set of versionCodes + notes published to a track.
Edit	An API transaction: opened, modified, validated and committed (or deleted).
Opt-in	The tester's explicit acceptance through the test link; it only counts if continuous.
Google Group	A `…@googlegroups.com` list that gives all its members access to the test.
Service account	A non-human Google Cloud identity the AI uses to call the API.
Android Publisher API v3	The official API for Play builds, tracks, listings and more.
Data safety	The form declaring what data the app collects and shares.
IARC	The age-rating system that produces the content rating.
Trader	A declaration that you act as a business seller (required by consumer-protection rules).
Promo code	A single-use code to get a paid app for free.
$0 sale	A temporary 100% discount on a paid app's price.
ASO	App Store Optimization: tuning the title and descriptions to show up in searches.
Managed publishing	A setting that holds approved changes until you click Publish.
Publishing overview	The Play Console page showing the actual review and publishing status.
Rollout	The percentage of users who receive a release (100% = completed).